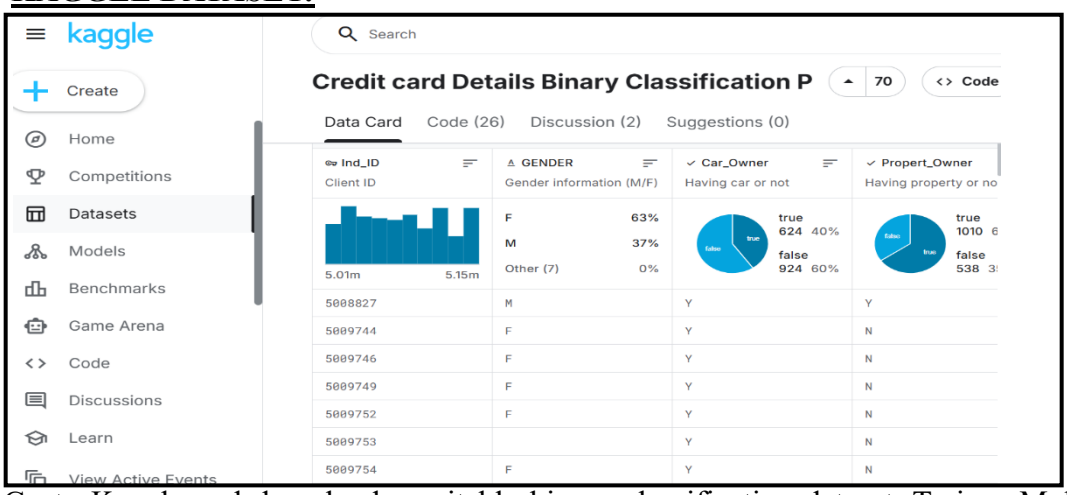
LAB # 04

OBJECTIVE:

To implement a Multilayer Perceptron for classification, understand its hidden layers and activation functions, and compare its performance with a simple perceptron.

LAB TASKS:

KAGGLE DATASET:



1. Go to Kaggle and download a suitable binary classification dataset. Train a Multilayer Perceptron (MLP) model on the dataset to predict the target variable. After training, evaluate the performance of the model using accuracy and test it on new unseen data samples.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
data = pd.read_csv("Credit_card.csv")
data = data.ffill()
if 'Ind_ID' in data.columns:
    data.drop(columns=['Ind_ID'], inplace=True)
categorical_cols = data.select_dtypes(include='object').columns
data = pd.get_dummies(data, columns=categorical_cols)
target_column = 'Car_Owner_Y' if 'Car_Owner_Y' in data.columns else 'Car_Owner'
X = data.drop(columns=[target_column])
y = data[target_column]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = MLPClassifier(hidden_layer_sizes=(64, 32),max_iter=500,random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
new_data = pd.DataFrame({
    'Gender': ['Male'],
    'Own_property': ['Yes'],
    'Type_Income': ['Commercial associate'],
    'Annual_income': [500000]})
new_data = pd.get_dummies(new_data)
new_data = new_data.reindex(columns=X.columns, fill_value=0)
new_data_scaled = scaler.transform(new_data)
prediction = model.predict(new_data_scaled)
print("Prediction:", prediction)
```

OUTPUT:

```
Accuracy: 0.9967741935483871
Prediction (Car Owner): [ True]
```

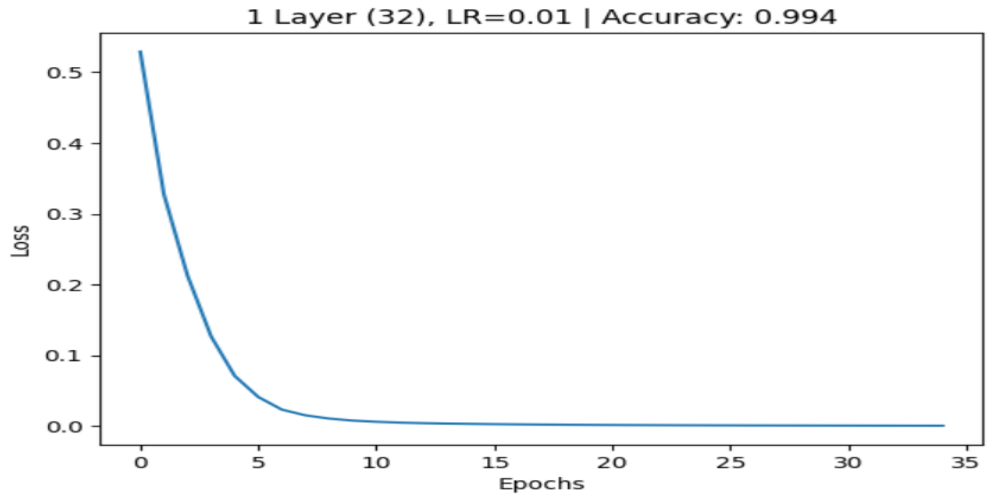
2. Modify the MLP model by changing the number of hidden layers and neurons (for example, using one hidden layer, two hidden layers, and different numbers of neurons). Train the model using different learning rates such as 0.01 and 0.1. Plot the loss versus epochs graph for each configuration and analyze how these changes affect the convergence speed and accuracy of the model.

CODE:

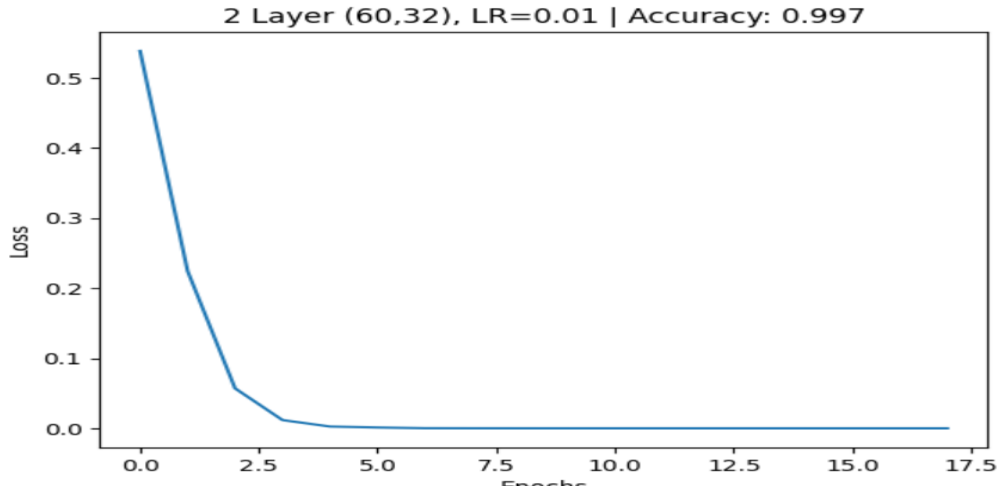
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
data = pd.read_csv("Credit_card.csv")
data = data.ffill()
if 'Ind_ID' in data.columns:
    data.drop(columns=['Ind_ID'], inplace=True)
categorical_cols = data.select_dtypes(include='object').columns
data = pd.get_dummies(data, columns=categorical_cols)
target_column = 'Car_Owner_Y' if 'Car_Owner_Y' in data.columns else 'Car_Owner'
X = data.drop(columns=[target_column])
y = data[target_column]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
configs = [
    {"name": "1 Layer (32), LR=0.01", "layers": (32,), "lr": 0.01},
    {"name": "2 Layer (60,32), LR=0.01", "layers": (60, 32), "lr": 0.01},
    {"name": "2 Layer (59,31), LR=0.1", "layers": (59, 31), "lr": 0.1},]
results = []
for config in configs:
    model = MLPClassifier(hidden_layer_sizes=config["layers"],learning_rate_init=config["lr"],
                          max_iter=200,random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results.append({
        "name": config["name"],
        "loss": model.loss_curve_,
        "accuracy": acc})
for res in results:
    plt.figure()
    plt.plot(res["loss"])
    plt.title(f"{res['name']} | Accuracy: {res['accuracy']:.3f}")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.show()
print("\nFinal Results:")
for res in results:
    print(res["name"], " Accuracy:", round(res["accuracy"], 4))
```

OUTPUT:

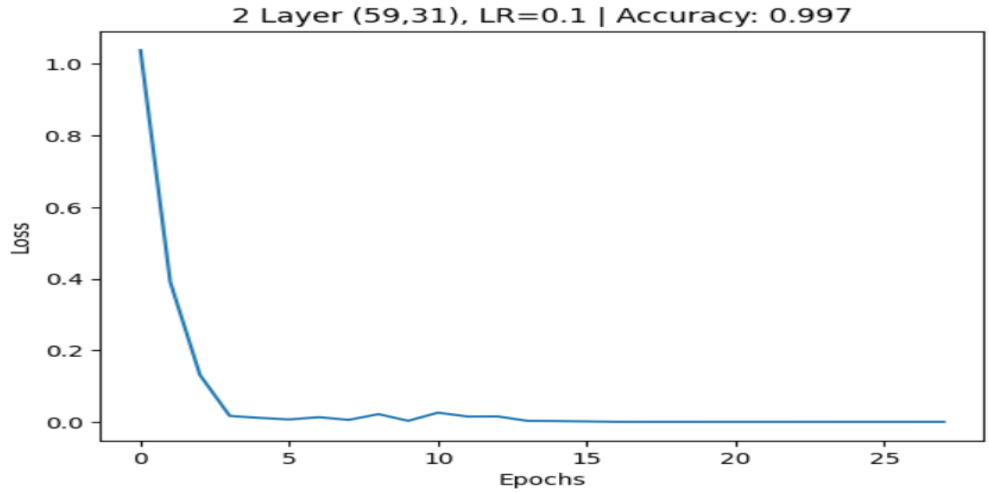
...



...



...




```
Final Results:
1 Layer (32), LR=0.01  Accuracy: 0.9935
2 Layer (60,32), LR=0.01  Accuracy: 0.9968
2 Layer (59,31), LR=0.1  Accuracy: 0.9968
```

ANALYZATION:

AFFECT OF HIDDEN LAYERS:

- **SINGLE HIDDEN LAYER AND 32 NEURONS:**
  - ✓ Model is simple.
  - ✓ Learns limited patterns.
  - ✓ May underfit the data.
  - ✓ Loss decreases slowly.
  - ✓ Accuracy is usually lower compared to deeper models.
- **TWO HIDDEN LAYER AND (64 , 32) NEURONS:**
  - ✓ Model becomes more powerful.
  - ✓ Learns complex non-linear patterns better.
  - ✓ Loss decreases more smoothly.
  - ✓ Usually gives higher accuracy

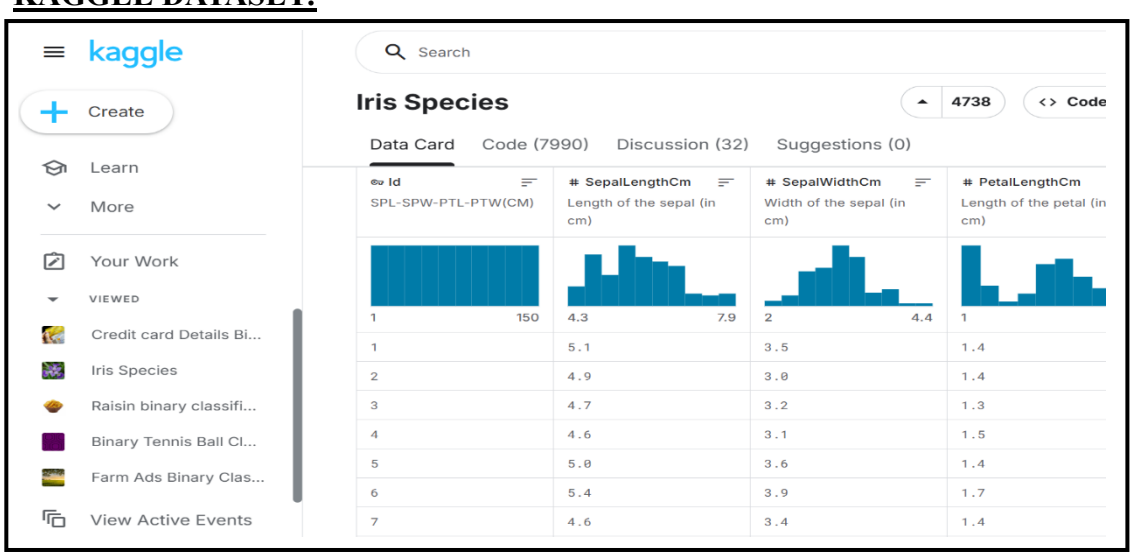
AFFECT OF LEARNING RATE:

- **LEARNING RATE lr = 0.01:**
  - ✓ Stable training.
  - ✓ Smooth and consistent decrease in loss.
  - ✓ Model converges properly.
  - ✓ Higher chance of better accuracy.
- **LEARNING RATE lr = 0.1:**
  - ✓ Faster learning initially.
  - ✓ Loss may fluctuate (unstable behavior).
  - ✓ Sometimes fails to converge properly.
  - ✓ Accuracy may decrease or become inconsistent.

BONUS TASKS:

Download a dataset containing three input features and three output classes from Kaggle or any CSV file. Implement a multi-class MLP model and train it on the dataset. Visualize the data in three dimensions and analyze how different features contribute to the classification.

KAGGLE DATASET:

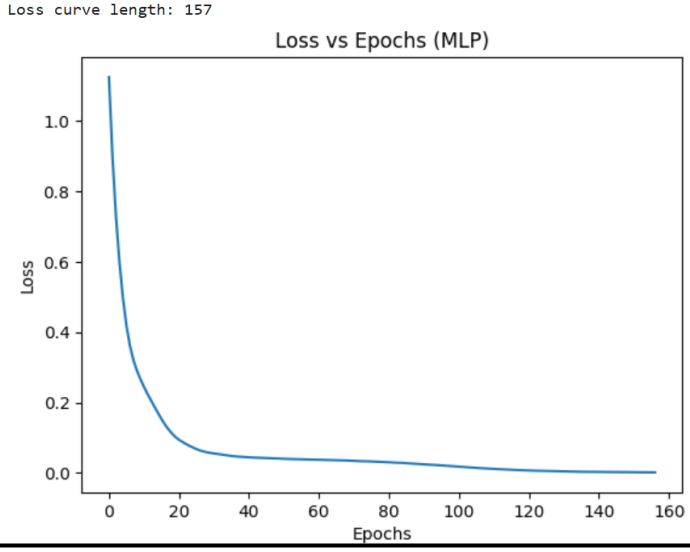


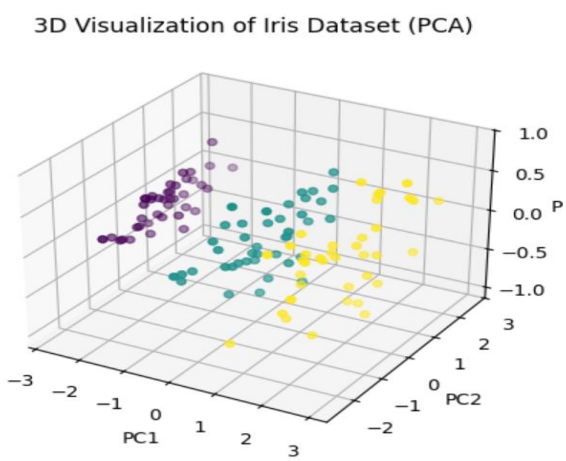
CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
data = pd.read_csv("Iris_dataset.csv")
data = data.drop(columns=["Id"])
le = LabelEncoder()
data["Species"] = le.fit_transform(data["Species"])
X = data.drop(columns=["Species"])
y = data["Species"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = MLPClassifier(hidden_layer_sizes=(60, 30),activation='relu',
                      learning_rate_init=0.01,max_iter=200,random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Loss curve length:", len(model.loss_curve_))
plt.figure()
plt.plot(model.loss_curve_)
plt.title("Loss vs Epochs (MLP)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
pca = PCA(n_components=3)
X_3d = pca.fit_transform(X_train)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(
    X_3d[:, 0],
    X_3d[:, 1],
    X_3d[:, 2],
    c=y_train,
    cmap='viridis')
ax.set_title("3D Visualization of Iris Dataset (PCA)")
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
plt.show()
```

OUTPUT:

```
Accuracy: 1.0
Loss curve length: 157
```





ANALYZATION:

The Multilayer Perceptron successfully classifies the Iris dataset into three classes with high accuracy. Feature analysis shows that petal length and width are the most important attributes for classification. The PCA-based 3D visualization clearly shows separable clusters, validating the effectiveness of the model.